

FDD3024: Module 2 Home Problem Set

Concurrency

Roberto Guanciale

ex 1: Peeterson Agorithm

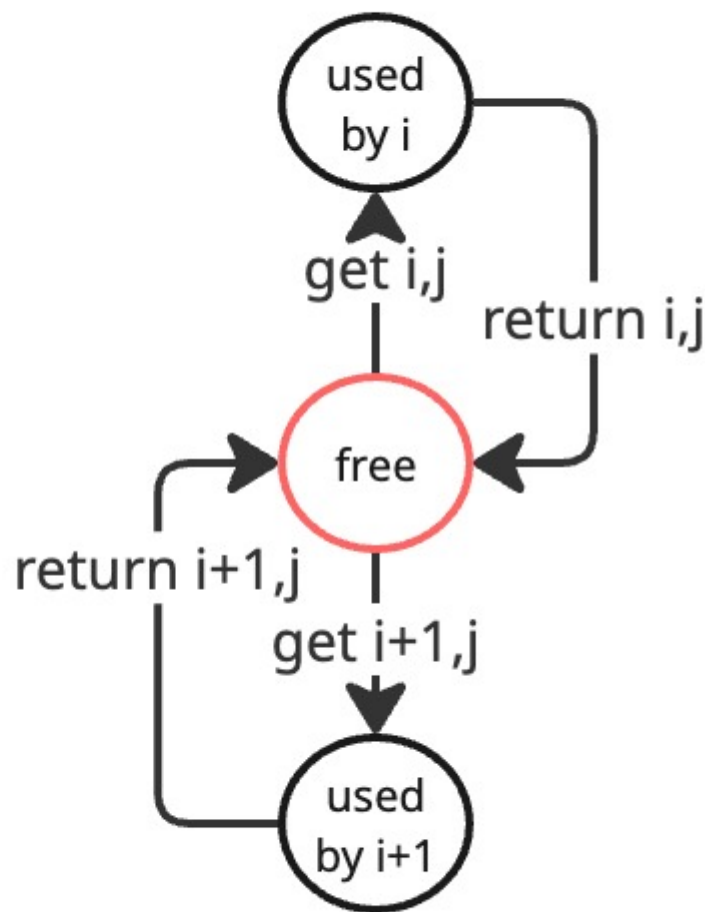
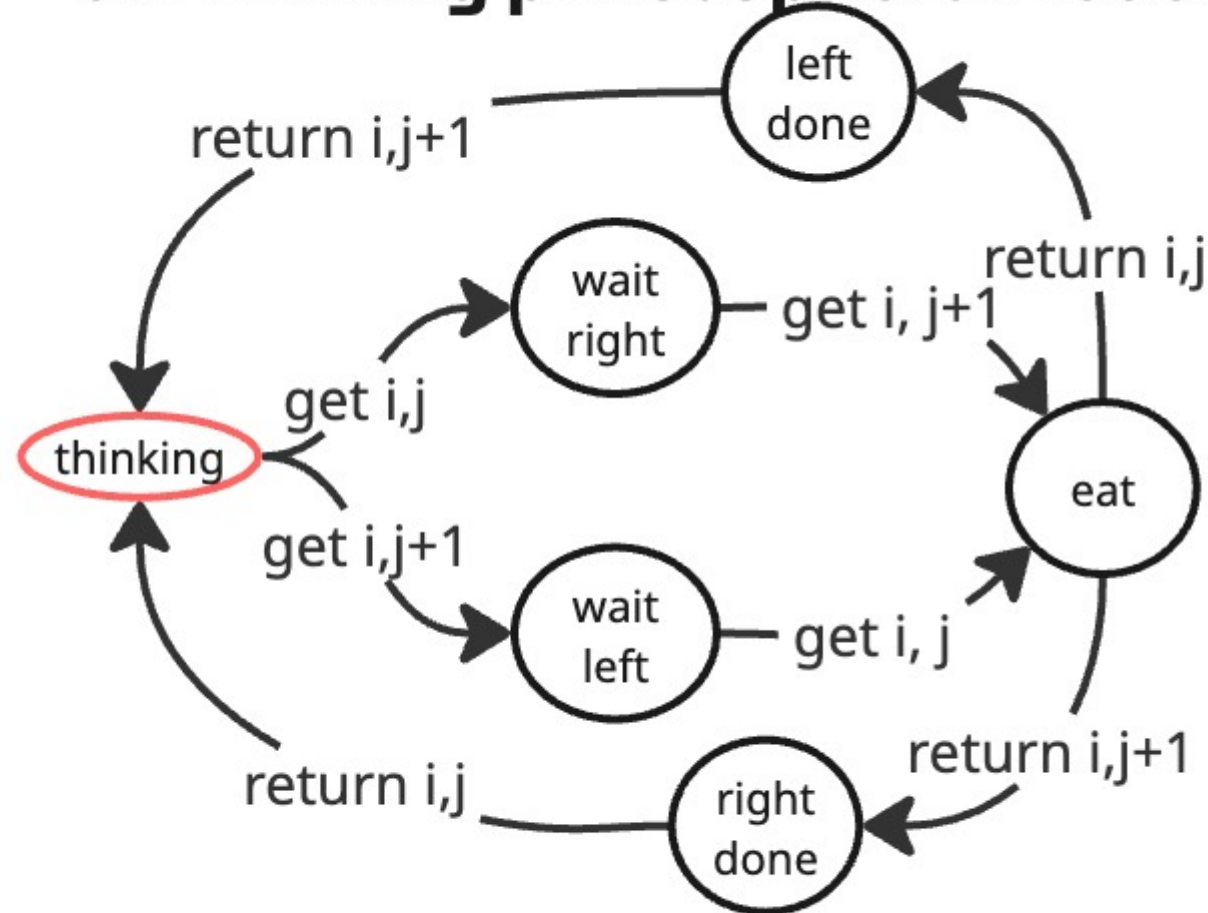
code executed by thread $i \in [0,1]$:

```
b[i] := true  
x := 1-i  
while x != i and b[1-i]:  
    skip  
// Critical section  
b[i] = false
```

$$\frac{p_2, m \rightarrow p'_2, m'}{p_1, p_2, m \rightarrow p_1, p'_2, m'}$$

Exercise, formulate mutual exclusion and prove it

ex 2: Dining philosophers: Deadlocks



Remove deadlock, formulate absence of deadlock, prove it for 2 philosophers

ex3: CCS examples

$$a.b.0 \parallel \bar{a}.0 \parallel \bar{a}.\bar{b}.0$$

$$(a.0 + b.0) \parallel \bar{a}.0$$

$$(a.0 + b.0) \parallel (\bar{a}.0 + \bar{b}.0)$$

$$(a.b.0 \parallel \bar{a}.0) \parallel \bar{a}.\bar{b}.0$$

$$\tau.(a.0 + b.0) \parallel (\bar{a}.0 + \bar{b}.0)$$

$$a.b.0 \parallel c.0$$

$$(\nu a)(a.b.0 \parallel \bar{a}.0) \parallel \bar{a}.\bar{b}.0$$

$$(\tau.a.0 + \tau.b.0) \parallel (\bar{a}.0 + \bar{b}.0)$$

Generate the LTS of at least 3 examples. For each transition explain the inference rules that you have applied

ex:4 LTS Isomorphism

Prove for at least 3 cases that there is an isomorphism between the LTSs of the left and right processes

$$a.b.0 + b.a.0$$

$$a.0 \parallel b.0$$

$$P + 0$$

$$P$$

$$P \parallel Q$$

$$Q \parallel P$$

$$P \parallel 0$$

$$P$$

ex:5 Bisimulation

Prove for at least 3 cases that the two processes are bisimilar

$$P \parallel Q \sim Q \parallel P$$

$$P + Q \sim Q + P$$

$$P \parallel (Q \parallel R) \sim (P \parallel Q) \parallel R$$

$$P + (Q + R) \sim (P + Q) + R$$

$$P \parallel 0 \sim P$$

$$P + 0 \sim P$$

$$P + P \sim P$$

ex:6 Bisimulation

Let $P \sim Q$, prove for at least 2 cases that the following processes are bisimilar

$$a.P \sim a.Q$$

$$P + R \sim Q + R$$

$$P \parallel R \sim Q \parallel R$$

$$(\nu a)P \sim (\nu a)Q$$

$$!P \sim !Q$$

FDD3024: Module 2 Home Problem Set

Weak Memory

Pablo Buiras and Mads Dam

Exercise 1 Assume an initial memory where X, Y and A are all set to 0. Consider the following threads:

<pre>T0: X = 1 if (Y = 0) { A = 2 }</pre>	<pre>T1: Y = 1 if (X = 0) { A = 3 }</pre>
---	---

For each of the following schedules, mark if they are allowed under Sequential Consistency (SC), Total Store Ordering (TSO), neither, or both. Indicate which reordering constraints they violate, if any.

1. T0:Read(Y,0), T1:Write(Y,1), T0:Write(X,1), T1:Read(X,0),
T0:Write(A,2), T1:Write(A,3)
2. T0:Write(X,1), T1:Write(Y,1), T0:Read(Y,1), T1:Read(X,1)
3. T0:Write(A,2), T0:Write(X,1), T0:Read(Y,0), T1:Write(Y,1),
T1:Read(X,1)
4. T0:Read(Y,0), T1:Read(X,0), T0:Write(X,1), T1:Write(Y,1),
T0:Write(A,2), T1:Write(A,3)
5. T0:Write(X,1), T0:Read(Y,0), T1:Write(Y,1), T1:Read(X,1),
T0:Write(A,2)

Exercise 2 Machines with relaxed memory consistency typically provide programmers with fence instructions to tighten the ordering of memory instructions. Insert MFENCE (memory fence) instructions in Peterson's algorithm to ensure its correct behavior on a multi-processor system that implements a write buffer with store bypass. Assume that the fence atomically flushes the write buffer. Use as few fence instructions as necessary.

Exercise 3 For each of the tests in Figure 1, Figure 2, and Figure 3, explain if the following outcomes are possible under TSO.

1. Test 1: $r1 = 1$ and $r2 = 0$
2. Test 2: $r2 = 0$ and $r3 = 0$
3. Test 3: $r0 = 0$ and $r2 = 0$

In all three cases, assume an initial state where $A = 0$ and $B = 0$.

$$\begin{array}{l|l} \text{XCHG}(A, r0) & r1 = B \\ B = 1 & r2 = A \end{array}$$

Figure 1: Test 1

$$\begin{array}{l|l} A = 1 & B = 1 \\ r0 = A & r1 = B \\ r2 = B & r3 = A \end{array}$$

Figure 2: Test 2

$$\begin{array}{l|l} \text{XCHG}(A, r0) & B = 1 \\ r1 = B & r2 = A \end{array}$$

Figure 3: Test 3

FDD3024: Module 2 Home Problem Set

Modal and Temporal Logic

Mads Dam

For the context of each exercise, see the corresponding lecture slides.

Positive Forms

Abbreviate

- $\phi_1 \wedge \phi_2 \equiv \neg(\neg\phi_1) \vee (\neg\phi_2)$
- $\diamond\phi \equiv \neg(\Box\neg\phi)$

Positive modal formulas:

$$\phi ::= x \mid \neg x \mid \phi \vee \phi \mid \phi \wedge \phi \mid \Box\phi \mid \diamond\phi$$

Exercise 2

1. Show that
 $w \models \diamond\phi$ if exists w' such that $w R w'$ and $w' \models \phi$
1. Show that each modal formula can be rewritten to positive form

Soundness and Completeness

Theorem 1 (Soundness of K)

If $\phi \in K$ then ϕ is valid

Exercise 3:

Prove theorem 1

Modal Expressiveness

Normal modal logic addresses all Kripke models
Sometimes we're interested in classes of models

Validity wrt class:

Let $\mathcal{C}$ be a class of Kripke models. $\models_{\mathcal{C}} \phi$ if for all models $M = (W, R, V) \in \mathcal{C}$ and $w \in W$, $w \models \phi$

Exercise 5:

Suggest formulas to be added to K in order to characterise the classes of models for which R is:

1. Reflexive
2. Transitive
3. Symmetric

More Bisimulation

Exercise 6: Reprove example 1 using theorem 3.

Bisimulations exist in many variants, strong, weak, branching, etc.

Generally, it is possible to devise logic characterisations similar to theorem 3 by applying modal logic to the appropriately adapted Kripke frames/transition systems.

Epistemic Logic

In epistemic logic usually use $K\phi$ in place of $\Box\phi$
Worlds can have multiple observers/agents, a , with different observation power, i.e. different L 's. So:

$$w \models K_a\phi \text{ if for all } w' \text{ if } w \sim_a w' \text{ then } w' \models \phi$$

Exercise 7:

What do these formulas mean?

- $K_a K_b x = 2, K_a(x = y \wedge \neg K_b(x = y))$
- $\neg K_a\phi, \neg K_a\phi \wedge \neg K_a\neg\phi$

Epistemic logic is a rich and interesting topic, but here it comes into its own only once we add dynamics

Exercise 8: Mutex

Assume there are 2 processes, P_l and P_r

State assertions:

- $tryCS_i$: Process i is trying to enter critical section
E.g. $tryCS_l: pc_l = l_4$
- $inCS_i$: Process i is inside its critical section
E.g. $inCS_l: pc_l = l_5 \vee pc_l = l_6$

Express the following properties in LTL:

1. Mutual exclusion, i.e. no two processes are in critical region at the same time
2. Responsiveness, i.e. that whenever a process tries to enter the critical region then it will some time in the future do so
3. That a process keeps trying until access is granted

Fairness Conditions

Weak transition fairness:

$$\bigwedge_{\alpha \in \Sigma} \neg FG(en_{\{\alpha\}} \wedge \neg exec_{\{\alpha\}})$$

Is this the same as

$$\bigwedge_{\alpha \in \Sigma} FGen_{\{\alpha\}} \Rightarrow GF exec_{\{\alpha\}} ?$$

Strong transition fairness:

$$\bigwedge_{\alpha \in \Sigma} GF en_{\{\alpha\}} \Rightarrow GF exec_{\{\alpha\}}$$

Weak process fairness:

$$\bigwedge_P \neg FG(en_P \wedge \neg exec_P)$$

Strong process fairness:

$$\bigwedge_P GF en_P \Rightarrow GF exec_P$$

(Many other variants are possible)

Exercise 9: Which implications hold between these five fairness conditions. Explain.

CTL, Semantics

...

$s \models AF\phi$ if for all w such that $w(0) = s$ exists $k \geq 0$ such that $w(k) \models \phi$

$s \models AG\phi$ if for all w such that $w(0) = s$, for all $k \geq 0$, $w(k) \models \phi$

$s \models A(\phi U \psi)$ if for all w such that $w(0) = s$, exists $k \geq 0$ such that $w(k) \models \psi$ and for all $i : 0 \leq i < k$, $w(i) \models \phi$

$s \models AO\phi$ if for all w such that $w(0) = s$, $w(1) \models \phi$

Exercise 10: Express the following properties in CTL:

1. No matter what the future might bring it is possible to win a million dollar
2. It is possible to win a million dollar any number of times, but it is also possible to never win anything